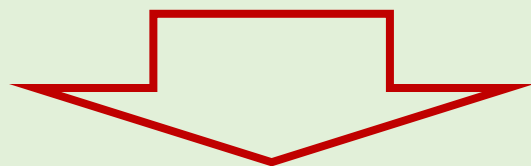
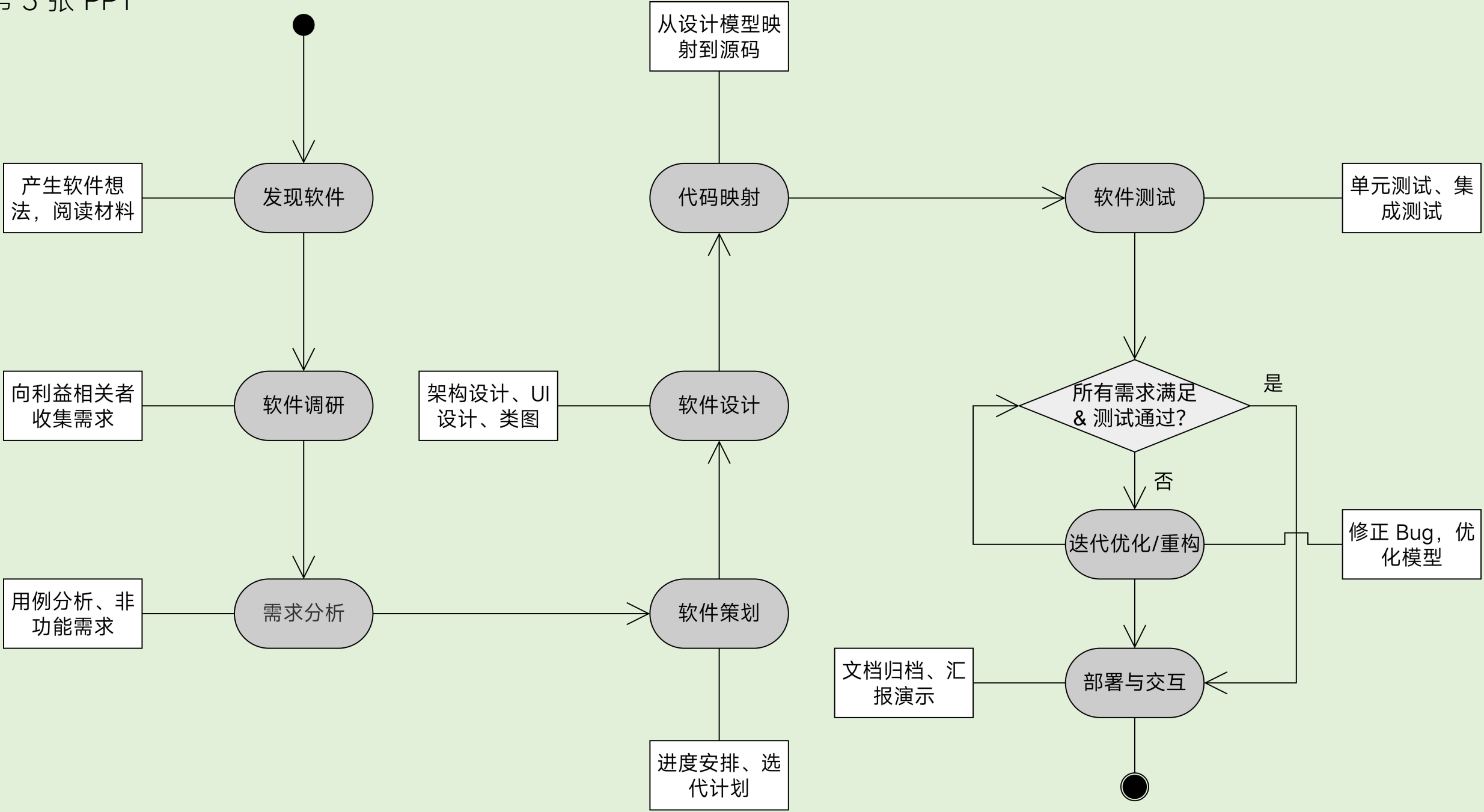


线性规划单纯形法求解器 设计与实现

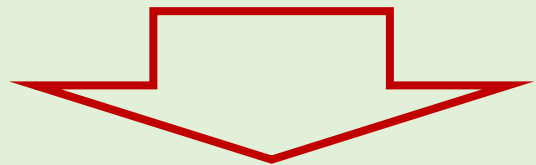
袁俊炫 2023141460166

线性规划单纯形法求解器 开发流程





第 1 步：发现软件



1.1 现状业务流程与痛点分析

故事背景：

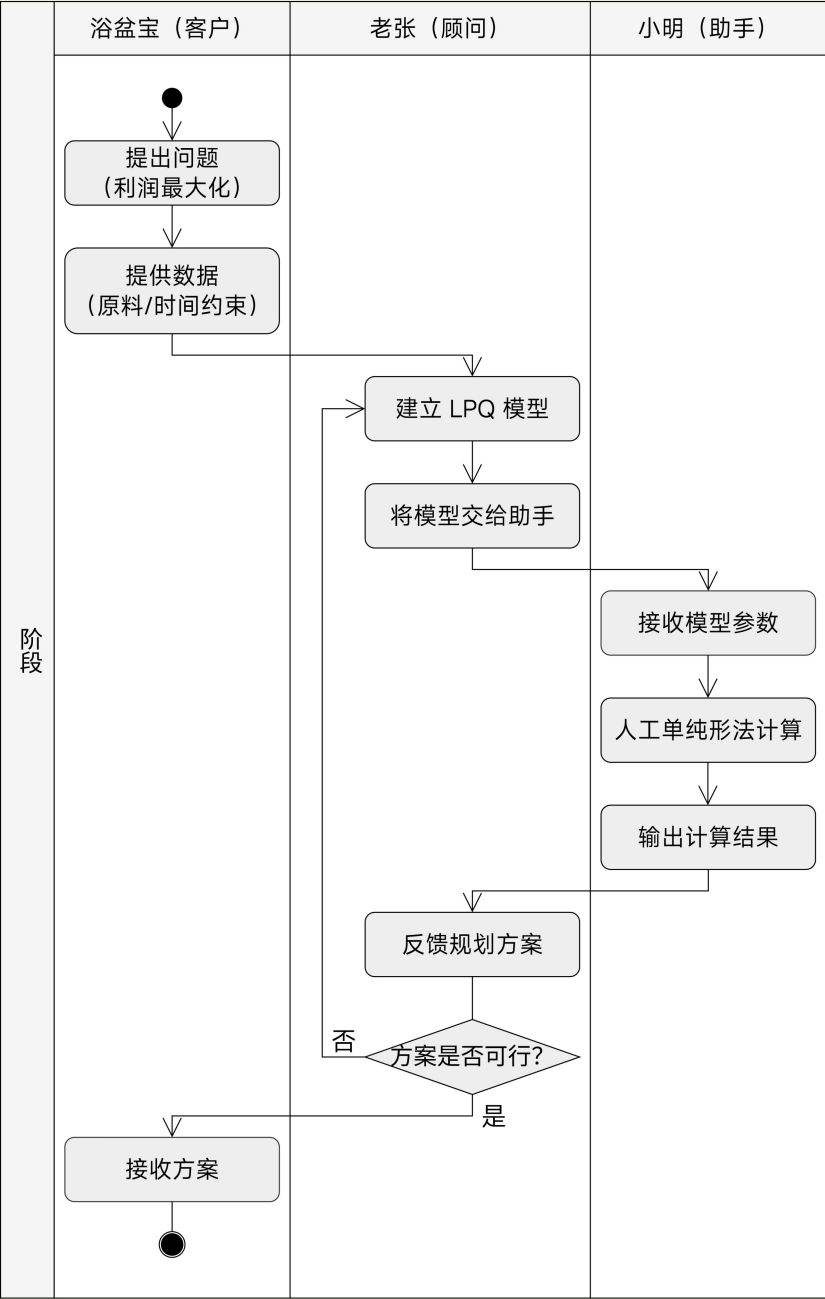
运筹学顾问老张负责为客户（浴盆宝公司）建立数学模型，助手小明负责后续的计算工作。

当前工作模式：

- 1. 老张与客户沟通，建立 LP（线性规划）模型。
- 2. 老张将模型参数交给小明。
- 3. 小明使用纸笔，通过单纯形法（Simplex Method）进行人工迭代计算。
- 4. 小明将计算结果反馈给老张。

核心痛点：

这是一个高人力依赖、低容错率且响应迟滞的串行化作业流程。



痛点：
手工计算耗时
易出错
重复劳动

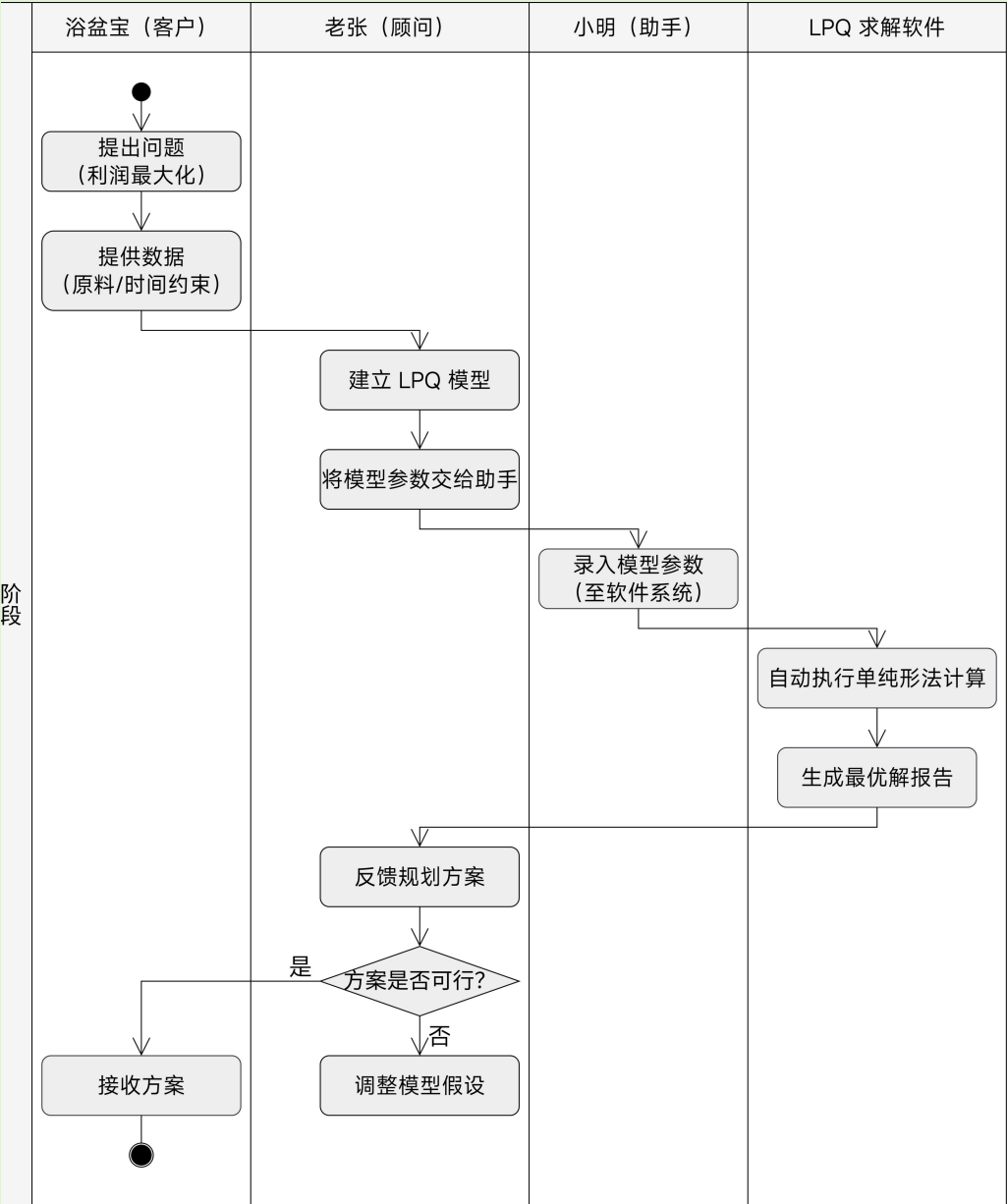
基于“运筹学家的故事”提取的现状业务流程泳道图（人工计算模式）

1.2 软件构想

软件名：
LPQ Assistant

最优先被照顾的涉众：
运筹学顾问（老张）、计算助手（小明）

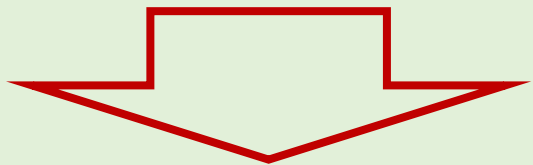
- 目标（度量指标）：
- 1. 效率指标：单纯形法计算耗时从人工的 > 30 分钟 缩短至 < 1 秒。
 - 2. 质量指标：计算结果准确率从 80% 提升至 100%（消除人工算术错误）。



改进价值：
增加 1 泳道
这就是职责重分配

引入“LPQ求解工具”后的改进业务流程泳道图（软件辅助模式）

第 2 步：软件调研



2.1 案例启发式访谈

访谈时间：项目启动初期

访谈对象：运筹学顾问（老张）、计算助手（小明）

调研问题	用户反馈
Q1: 期望的“LPQ计算”服务（系统用例）的交互序列是怎样的？	A1: 调研并整理数据得到： 用例名：计算 LPQ 用例概述：计算 LPQ 是从 LPQ 数学模型给出后开始，到系统输出规划方案的一个过程。 参与者：规划者（老张 / 小明） 前置条件：LPQ 数学模型已经建立。 基础路径： 1. 规划者提请计算（提交模型参数）。 2. 系统执行单纯形法计算。 3. 系统反馈规划方案（最优解）。 后置条件： 电脑输出 LPQ 规划方案。

调研问题

用户反馈

A2：调研获得数据，整理得到：

Q2：输入 LPQ 数学模型的特征调研。

LPQ

问题陈述：

导入

G.A.Tanner公司生产的一种产品是一种特别的玩具,预计每一种这种玩具可以带来3美元的利润.由于这种玩具的需求量很大,因此经理希望能够从当前每天生产1000件产品的基础上增加产量.但是,由于从供应商那里获得部件A和B的数量有限,所以很难提高产量。每个玩具需要两个A部件和一个B部件。
供应商对于A部件的供应数量只能从现在的每天2000个提高到3000个;而对于B部件的供应数量仍然为1000个.由于目前没有办法找到其他提供这两种部件的供应商,所以经理决定自己生产这两种部件来满足需求.但是生产时由于某种原因A和B必须等量同时生产.按照估计,如果公司自己生产这两种部件的话,因为A和B同时生产,那么每生产一组A和B,所花费的总成本较之从供应商那里购买花费的总成本相比,每一组A和B都会高出2.5美元.因此,经理想要得到玩具和两种部件各自产量分别都是多少,来使利润实现最大化。

建模向导:

目标: ☒ max ☐ min z

竞争性活动: +

玩具[可编辑]	x ₁ [默认, 可编辑]
A+B [可编辑]	x ₂ [默认, 可编辑]

资源: +

A[可编辑]	≤ [默认]
B [可编辑]	≤ [默认]

参数:

资源	单位竞争性活动		资源限制
	玩具x ₁	A+B x ₂	
A	2	-1	3000
B	1	-1	1000
单位利润	3	-2.5	
【参数均可编辑】			

【建模】

建模

数学模型

$$\begin{aligned} \max z &= 3x_1 - 2.5x_2 \\ \text{s.t.} \quad &\begin{cases} 2x_1 - x_2 \leq 3000 \\ x_1 - x_2 \leq 1000 \\ x_1 \geq 0, x_2 \geq 0 \end{cases} \end{aligned}$$

计算

≤ [默认]

≤
<
=
≥
>

【下拉菜单选择】

规划方案

max z=...;

<第一种竞争性活动>x₁=... ,
<第二种竞争性活动>x₂=... ,
..... ,
<第一种竞争性活动>x_n=...;

<第一种资源>x_{n+1}=... ,
<第二种资源>x_{n+2}=... ,
..... ,
<第三种资源>x_{n+m}=... 。

关闭

【计算弹出】

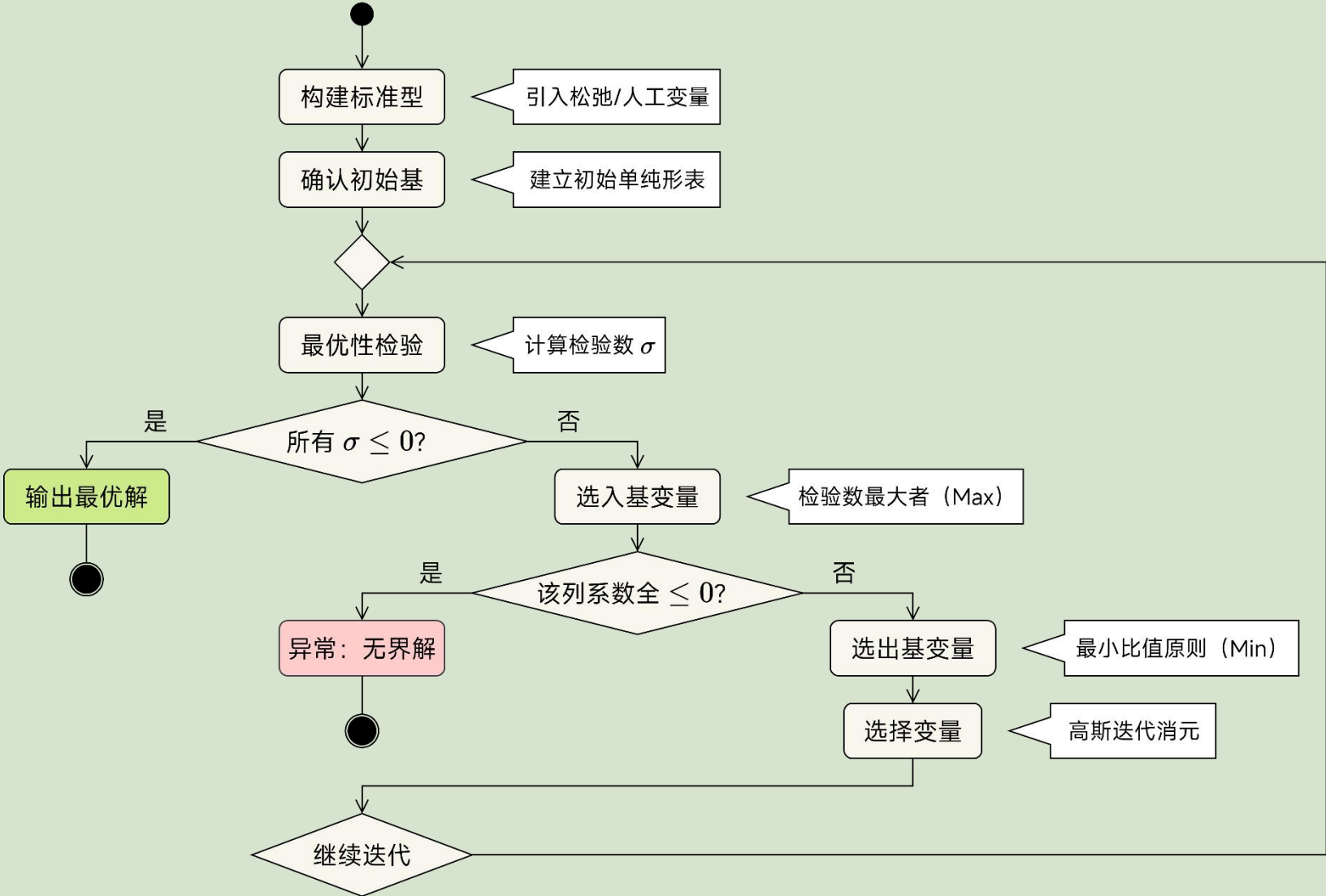
调研问题	用户反馈
Q3：怎样“提请”？	A3：用户希望通过一个“建模向导”来完成提请，具体步骤如下： 1. 问题陈述：首先输入业务场景描述（备注信息）。 2. 向导式配置（核心步骤）： • 定目标：选择最大化（Max）还是 最小化（Min）。 • 定变量：定义竞争性活动（如：生产 A 产品、B 产品）。 • 定约束：定义资源限制（如：原料、时间），并设定约束方向（$\leq$，$<$，$=$，$>$，$\geq$）。 3. 触发动作： • 点击“建模”：系统自动生成数学模型公式。 • 点击“计算”：系统执行算法并返回结果。

调研问题	用户反馈
Q4: LPQ 建模方法的学习及单纯形算法的学习。	A4: 根据对运筹学方法的调研，软件核心算法应基于人工推导所得的逻辑规则。 转化标准形 (Standardization) : - 将线性规划问题转化为目标函数求极大、约束全为等式、右端项非负、变量非负的标准形式。 - 引入松弛变量 / 剩余变量 / 人工变量构造初始单位矩阵。 确定初始基可行解 (Initialization) : 以单位矩阵对应的变量为基变量，非基变量设为 0，解出基变量值，得到初始基可行解。 最优性检验 (Optimality Test) : 计算非基变量的检验数 (σ_j)，若所有检验数 ≤ 0，当前解为最优解，停止计算；否则进入迭代。 迭代优化 (Iteration) : 选换入变量: 选取检验数最大（最正）的非基变量作为换入变量。 选换出变量: 按最小比值原则 (θ)，在换入变量列的正系数中，计算右端项与系数的比值，选取比值最小的对应基变量作为换出变量。 高斯消元: 以换入变量与换出变量的交叉元素为主元，进行初等行变换，使主元变为 1，该列其他元素变为 0，得到新的基可行解。 重复步骤 3-4: 直至所有检验数 ≤ 0，（得到最优解）或发现无界解/无可行解。

调研问题

用户反馈

Q4: LPQ
建模方法
的学习及
单纯形算
法的学习。



单纯形算法核心流程

调研问题	用户反馈
Q5：LPQ 数学模型的一般形式及规划方案的输出形式。	A5：调研发现，软件需具备将自然语言转化为标准数学模型的能力，并提供可视化的结果反馈。 1. LPQ 数学模型的一般形式： 单纯形法求解要求模型满足“标准型”： • 目标函数： 统一转化为求极大值 $\max Z$。 • 约束条件： 所有不等式需转化为等式（引入松弛/人工变量），且右端常数项 $b_i \geq 0$。 • 变量约束： 所有决策变量 $x_j \geq 0$。 $\max Z = \sum_{j=1}^n c_j x_j$ $\text{s. t. } \sum_{j=1}^n a_{ij} x_j = b_i \quad (i = 1, \dots, m)$ $x_j \geq 0, \quad b_i \geq 0$

调研问题	用户反馈
Q5: LPQ 数学模型的一般形式及规划方案的输出形式。	2. 规划方案的输出形式： 用户期望获得一份“多层次、可视化”的决策报告： • 最优目标值：明确显示最大利润或最小成本（如 $\max Z = \dots$）。 • 最优生产计划：清晰列出每个决策变量（x_i）的最佳取值。 • 资源约束分析：显示松弛变量（x_{n+i}）验证每种资源的消耗情况。 规划方案 $\max z = \dots;$ <第一种竞争性活动> $x_1 = \dots$, <第二种竞争性活动> $x_2 = \dots$, , <第n种竞争性活动> $x_n = \dots;$ <第一种资源> $x_{n+1} = \dots$, <第二种资源> $x_{n+2} = \dots$, , <第m种资源> $x_{n+m} = \dots$ 。 关闭

调研问题	用户反馈
Q6：单机系统还是分布式系统？	A6：采用分布式系统（Distributed System）。 1. 计算性能需求：处理“大规模、高并发”的数学计算时，单机性能存在瓶颈。分布式架构可利用计算服务器集群，突破单机限制。 2. 多人协作需求：需要满足多位运筹师（不仅是老张和小明）独立、并行地进行建模和运算，支持数据共享。 3. 扩展性：未来可能处理更复杂的数学问题（不仅仅是 LPQ），分布式架构更易于扩展计算能力。

2.2 调研总结与需求规格确立

业务与交互:

- **核心流程**: 明确了“规划者提请计算 → 系统计算 → 反馈方案”的闭环路径。
- **输入模式**: 采用“**向导式建模**”，支持业务背景输入、目标配置 (Max/Min)、变量/约束定义三步操作。

核心算法与模型:

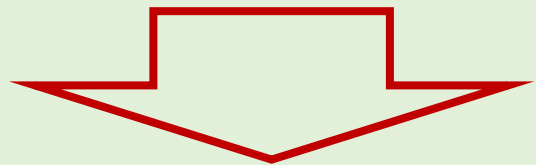
- **数学内核**: 掌握了 LPQ 建模的一般形式 (目标函数、约束条件、非负约束)。
- **算法引擎**: 确立了以单纯形算法 (Simplex Method) 为核心, 支持大 M 法处理人工变量的计算逻辑。
- **输出规范**: 输出包含最优目标值、变量解及约束验证结果。

架构与非功能需求:

- **技术架构**: 选用**分布式系统架构**, 以支持大规模计算和高并发访问, 避免单机瓶颈。
- **质量属性**: 高效性 (毫秒级响应); 可靠性 (精度高, 具备校验机制); 易用性 (提供可视化向导界面); 扩展性 (支持 Excel 兼容与跨设备访问)。



第 3 步：需求分析

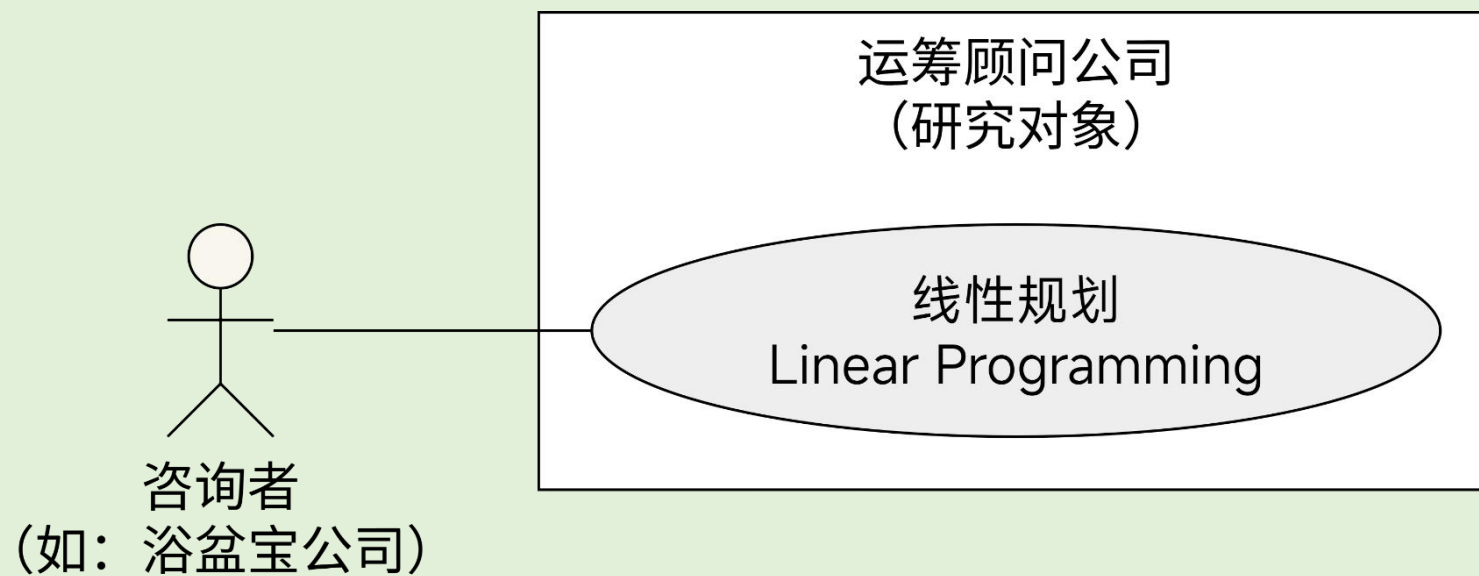


3.1 业务用例建模

通过问：“**运筹顾问公司（上下文系统）** 能够向外提供什么服务呢？”，然后，我们实施软件调研，向与所研究组织内部的涉众（老张）访谈。

可以直接获得关于该问题的答案。因为，“**线性规划**”这一业务用例代表了该研究对象的**本质价值**，很难改变，且概念的内涵非常明确。

整理并分析这些信息，提取出其中的关键概念：用例、参与者和主题，并用 UML 用例图描述：**<运筹顾问公司>** 能够向外部 **<咨询者>** 提供 **<线性规划>** 服务。



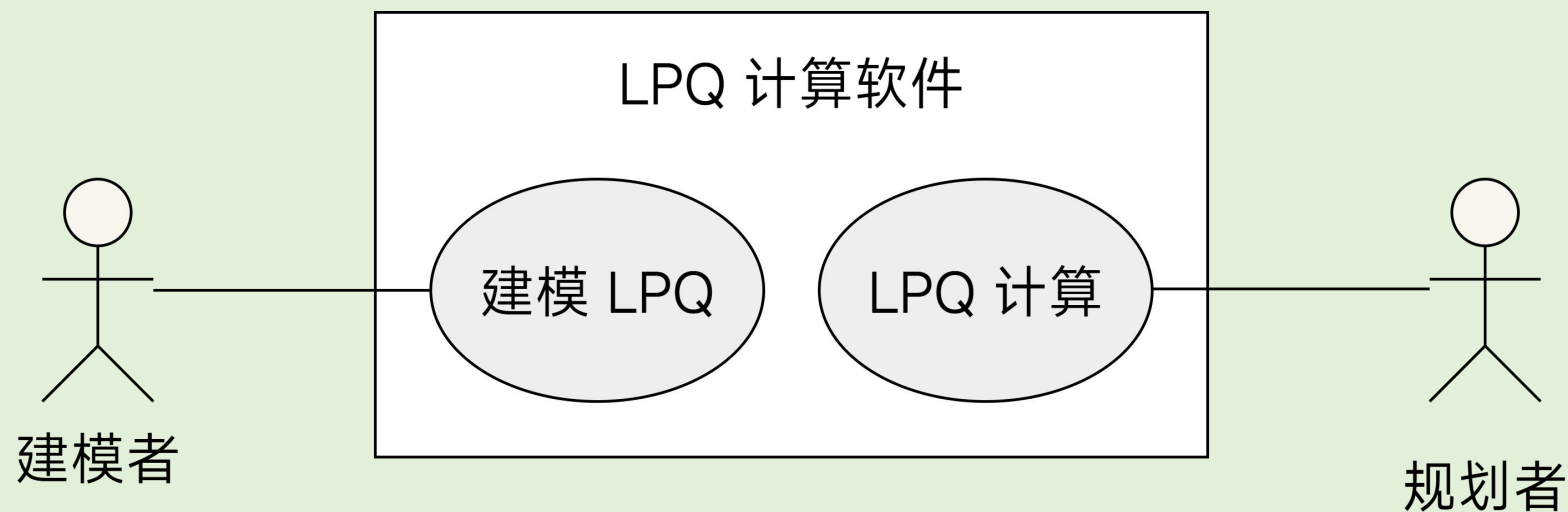
业务用例图

3.2 系统用例建模

基于前期的需求调研（Q1-Q3），识别系统参与者与功能边界，构建系统的用例模型。

采用 UML 用例分析法（Use Case Analysis）。通过识别**参与者（Actor）**和**用例（Use Case）**，界定系统边界；利用**包含（Include）**关系抽象出通用的算法逻辑（如单纯形计算）。

得到了**LPQ 计算软件的系统用例图**。明确了“建模LPQ”和“LPQ计算”是系统的核心功能点，且规划者是主要交互对象。



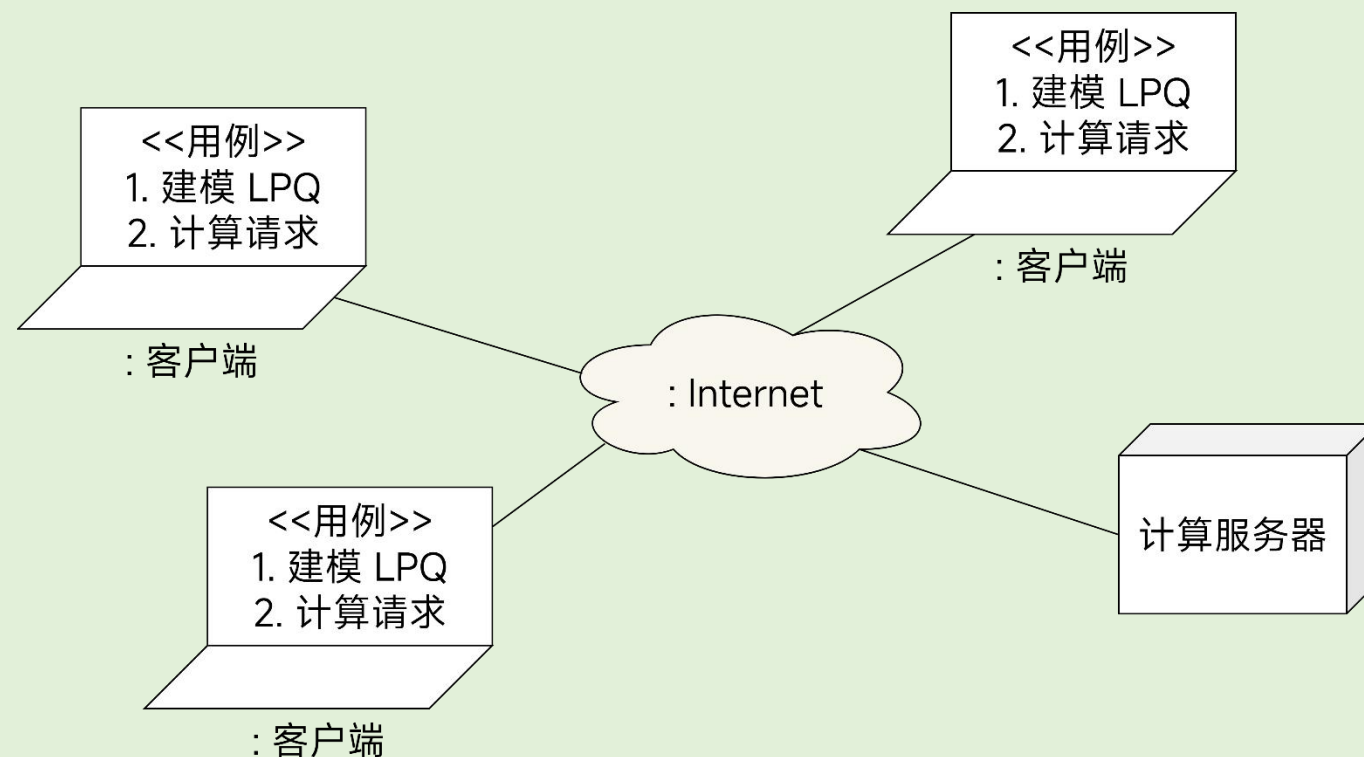
系统用例图

3.3 系统部署图

基于调研中确定的“分布式系统”和“多用户并发”需求，对系统的物理运行环境进行了规划。

采用了**分布式架构设计（Distributed Architecture）**方法，具体选用了**C/S（客户端/服务器）**架构风格，将展现层与计算层物理分离。

构建了**LPQ 上下文系统部署图**，明确了客户端、服务器的物理分布及通信关系。



系统部署图

3.4.1 建模功能规格化与交互设计

对核心“建立 LPQ 模型”用例进行了完整规格化，拆解了从问题输入到模型生成的全流程。

采用**用例规约**方法，通过**结构化模板**明确定义参与者与系统交互边界，清晰描述前置条件、主成功场景及后置条件，确保需求无歧义。

产出**标准用例规约表**，为 UI 设计与后台逻辑实现提供精准业务依据。

项目	说明
用例名称	建立 LPQ 模型（Build LPQ Model）
用例概述	录入线性规划问题描述，并辅助生成其标准数学模型的过程。
参与者	建模者 / 运筹规划师
前置条件	系统已启动，处于“新建项目”就绪状态。
交互流程	1. [用户] 输入线性规划问题的自然语言描述。 2. [系统] 解析描述，显示建模向导界面。 3. [用户] 选择决策目标（Max 或 Min）。 4. [系统] 确认目标，并请求定义变量。 5. [用户] 输入竞争性活动名称。 6. [系统] 自动生成变量符号（$x_1 \dots x_n$），并展开系数矩阵表格。 7. [用户] 在表格中填入技术系数（a_{ij}）与限制量。 8. 系统响应：系统自动生成并显示标准的 LPQ 数学模型公式。
后置条件	系统内存中生成了一个包含目标函数和约束条件的模型对象，可随时进行计算。

3.4.2 运算功能规格化与算法调用

对核心“运算求解”功能进行完整规格化，清晰定义了从用户触发计算到获取结果的完整流程。

采用**用户视角**设计交互逻辑，聚焦输入参数与输出方案，将复杂**算法封装**为系统内部能力。

明确“标准化→迭代→求解”的执行路径，并建立无解/无界等异常情况的反馈机制，为算法开发提供精准指导。

项目	说明
用例名称	运算求解 (Execute Calculation)
用例概述	系统调用单纯形法引擎对已建立的数学模型进行求解的过程。
参与者	LPQ 规划者
前置条件	LPQ 数学模型已建立完毕，且通过了数据完整性校验。
交互流程	1. [系统] (前置) 界面显示已建立的数学模型。 2. [用户] 点击“开始计算”按钮。 3. [系统] 响应计算请求（内部处理）： - 将模型转化为标准型（引入松弛变量）； - 构造初始单纯形表； - 执行迭代运算直至满足最优性检验。 4. [系统] 计算完成，生成“规划方案报告”，显示最优值（<i>Z</i>）及变量解。
后置条件	系统显示计算结果，模型进入“已计算”状态，可支持数据导出或重新修改。

3.5 定义数字字典

提取核心数据对象（决策变量、约束条件、目标函数），明确定义名称、类型及校验规则。

实现业务概念到数学模型的精准映射：将“鸭子数量”等业务术语抽象为标准化变量，确保界面控件与数据定义严格对应。

产出无歧义的《LPQ 系统数据字典》，为数据库设计与代码实现提供统一标准。

数据项名称	别名	数据类型	长度/范围	说明
模型 ID	Model_ID	String	32 chars	唯一标识一个线性规划模型
优化目标	Objective	Enum	{Max, Min}	目标函数的优化方向
决策变量	Variables	Array	$x_1 \dots x_n$	需决策的未知数集合，非负
系数矩阵	Coeff_Matrix	Double[][]	$m \times n$	约束方程组的左端系数 (A_{ij})
右端常数	RHS_Value	Double[]	m	资源限制量 (b_i)
约束符号	Operator	Enum	{ $\leq$, $=$, $\geq$ }	约束条件的关系运算符
最优解状态	Status	Int	0: 求解中, 1: 最优, 2: 无界, 3: 无解	算法计算后的返回状态码

3.6 页面原型设计

遵循界面设计三大黄金准则，采用“建模向导”引导用户分步定义目标、变量与约束，平衡规范性与灵活性。可视化矩阵表格降低录入难度，实时生成数学模型。计算结果在独立窗口清晰呈现，助力高效决策。

LPQ

问题陈述:

导入

建模向导:

目标: ☒ max ☐ min z

竞争性活动: +

玩具[可编辑]	x_1 [默认, 可编辑]
A+B [可编辑]	x_2 [默认, 可编辑]

资源: +

A[可编辑]	$\leq$ [默认]
B [可编辑]	$\leq$ [默认]

参数:

资源	单位竞争性活动		资源限制
	玩具 x_1	A+B x_2	
A	2	-1	3000
B	1	-1	1000
单位利润	3	-2.5	

【参数均可编辑】

数学模型

建模

建模

$$\begin{aligned} \max z &= 3x_1 - 2.5x_2 \\ \text{s.t.} \quad &\begin{cases} 2x_1 - x_2 \leq 3000 \\ x_1 - x_2 \leq 1000 \\ x_1 \geq 0, x_2 \geq 0 \end{cases} \end{aligned}$$

计算

【下拉菜单选择】

$\leq$ [默认]

$\leq$

$<$

$=$

$\geq$

$>$

规划方案

$\max z = \dots;$

$<\text{第一种竞争性活动}> x_1 = \dots,$

$<\text{第二种竞争性活动}> x_2 = \dots,$

$\dots,$

$<\text{第一种竞争性活动}> x_n = \dots;$

$<\text{第一种资源}> x_{n+1} = \dots,$

$<\text{第二种资源}> x_{n+2} = \dots,$

$\dots,$

$<\text{第三种资源}> x_{n+m} = \dots。$

关闭

3.7 定义非功能性需求

《需求（规格）说明书》

[子条目] 易用性

[命名] 智能纠错与可视化反馈

[概述] 系统应具备智能语法检查机制，对用户输入的数学表达式进行实时校验。若检测到括号不匹配、变量名冲突或非法字符，应立即在输入框旁显示红色高亮提示，并给出具体修改建议。

[涉众故事] 小明在录入一个复杂的约束方程时，不小心少打了一个乘号。系统没有报错崩溃，而是在输入框下方弹出提示：“检测到语法错误：第 3 行缺少运算符”，小明根据提示迅速修正了错误。

[子条目] 高效性

[命名] 大规模矩阵秒级求解

[概述] 针对包含 1000 个变量和 500 个约束条件的中型线性规划问题，求解引擎应在 **2 秒内**完成单纯形法迭代计算并输出结果。支持多线程并发计算，避免界面卡顿。

[涉众故事] 运筹顾问老张需要在会议上实时演示方案。他输入了一个庞大的物流优化模型，点击“计算”后，系统仅用 1.5 秒就给出了最优解，让客户对方案的响应速度印象深刻。

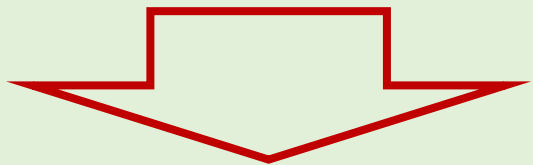
[子条目] 可扩展性

[命名] 算法策略插件化

[概述] 系统的核心计算模块应采用**策略模式（Strategy Pattern）**设计，允许在不修改核心代码的情况下，动态扩展新的求解算法（如：两阶段法、对偶单纯形法、内点法）。

[涉众故事] 一年后，公司业务扩展，需要处理一种特殊的退化问题。开发人员不需要重写整个软件，只需编写一个新的算法类并配置到系统中，老张就可以在界面上下拉选择“对偶单纯形法”来解决新问题。

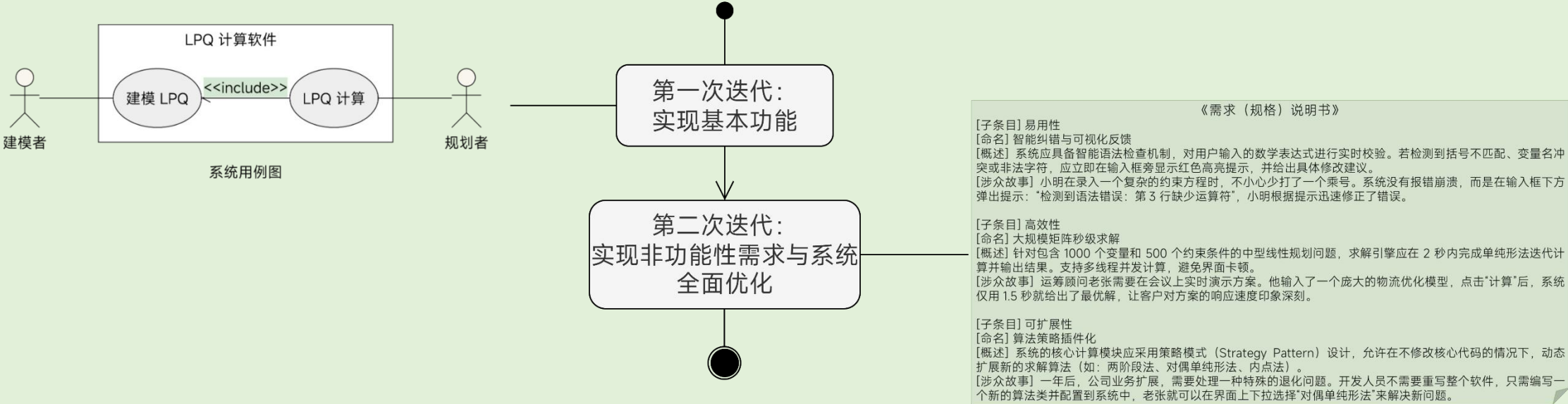
第 4 步：软件策划



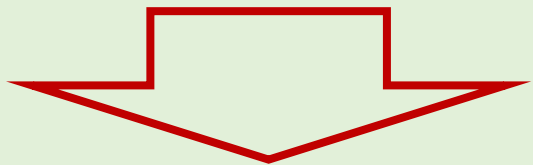
4 软件策划

基于“先核心后辅助、先功能后质量”的原则，制定了两阶段迭代开发计划：

- 1. **第一次迭代（MVP 阶段）**：我们将优先打通“建模 LPQ”到“LPQ 计算”的主流程，快速交付基础版本，解决用户最迫切的计算需求。
- 2. **第二次迭代（完善阶段）**：我们将重点部署易用性（智能纠错）、高效性（并行计算）和扩展性（策略模式），全面提升软件质量，将软件从“能用”提升为“好用”，满足非功能性需求（QAR）。



第 5 步：软件设计



5.1 从涉众故事到用例规约

通过“运筹学家的故事”，识别了**建模者（老张）**和**操作者（小明）**的关键交互场景，分析**非结构化描述**（如“输入约束”、“求解模型”），抽象出**系统核心功能边界**，并转化为**结构化用例规约**，明确前置条件、交互步骤及后置状态。

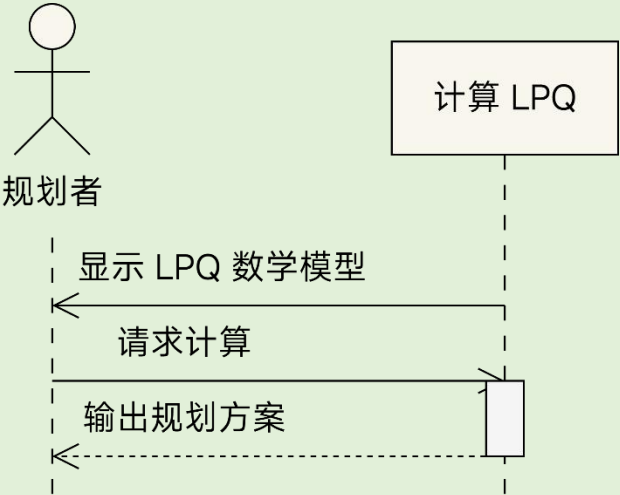
项目	说明
用例名称	建立 LPQ 模型 (Build LPQ Model)
用例概述	录入线性规划问题描述，并辅助生成其标准数学模型的过程。
参与者	建模者 / 运筹规划师
前置条件	系统已启动，处于“新建项目”就绪状态。
交互流程	1. [用户] 输入线性规划问题的自然语言描述。 2. [系统] 解析描述，显示建模向导界面。 3. [用户] 选择决策目标 (Max 或 Min) 。 4. [系统] 确认目标，并请求定义变量。 5. [用户] 输入竞争性活动名称。 6. [系统] 自动生成变量符号 ($x_1 \dots x_n$)，并展开系数矩阵表格。 7. [用户] 在表格中填入技术系数 (a_{ij}) 与限制量。 8. 系统响应：系统自动生成并显示标准的 LPQ 数学模型公式。
后置条件	系统内存中生成了一个包含目标函数和约束条件的模型对象，可随时进行计算。

项目	说明
用例名称	运算求解 (Execute Calculation)
用例概述	系统调用单纯形法引擎对已建立的数学模型进行求解的过程。
参与者	LPQ 规划者
前置条件	LPQ 数学模型已建立完毕，且通过了数据完整性校验。
交互流程	1. [系统] (前置) 界面显示已建立的数学模型。 2. [用户] 点击“开始计算”按钮。 3. [系统] 响应计算请求 (内部处理)： - 将模型转化为标准型 (引入松弛变量)； - 构造初始单纯形表； - 执行迭代运算直至满足最优性检验。 4. [系统] 计算完成，生成“规划方案报告”，显示最优值 (Z) 及变量解。
后置条件	系统显示计算结果，模型进入“已计算”状态，可支持数据导出或重新修改。

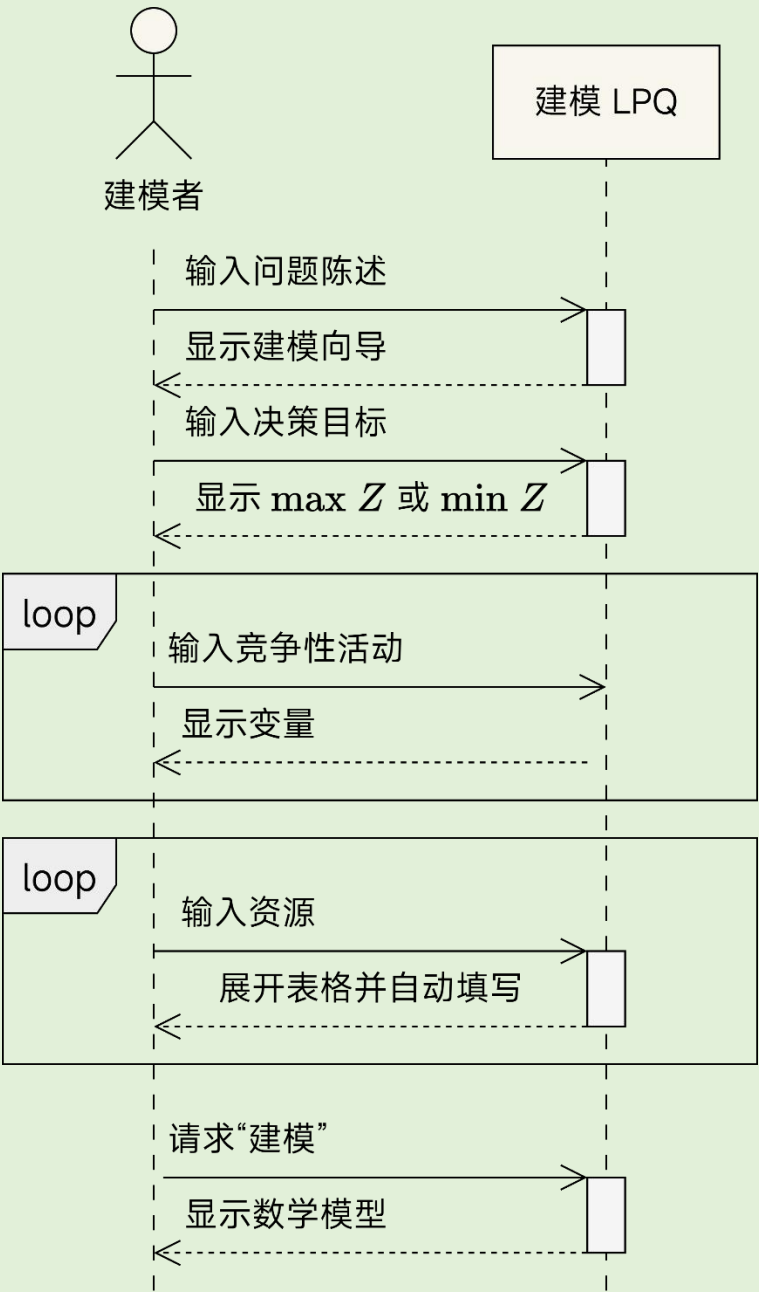
5.2 系统顺序图

依据用例规约，将核心场景映射为可视化的**系统顺序图**。通过定义精确的**系统消息**（如**输入决策目标**、**请求计算**），清晰界定了系统的黑盒边界。

这不仅规范了**交互接口**，更通过梳理消息传递逻辑，为后续从动态行为中**抽象出具体的类与方法**提供了直接依据。



计算 LPQ 顺序图



建模 LPQ 顺序图

5.3 领域类图

业务与交互：

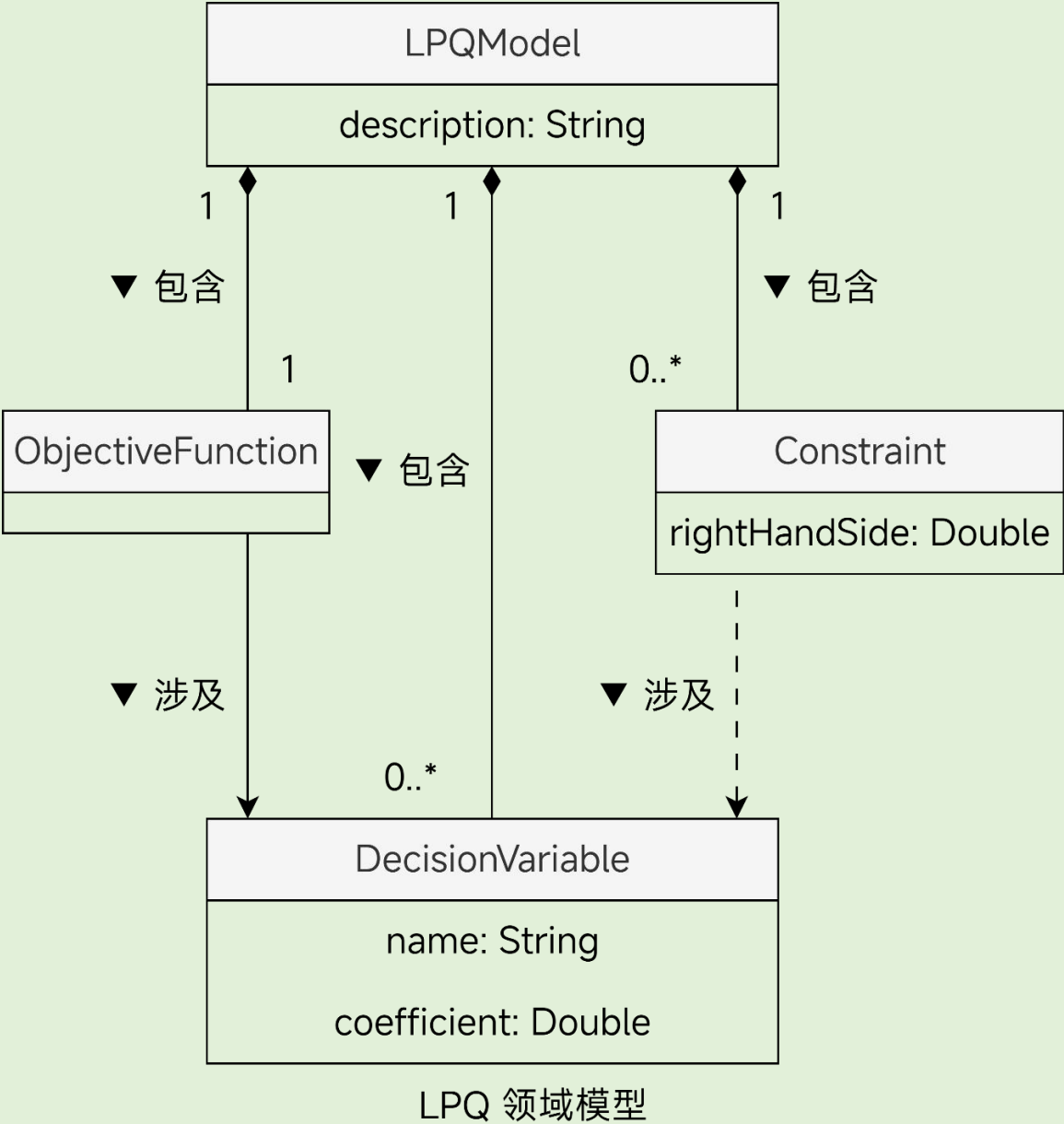
采用“名词短语识别法”，依据《分类规则表》（参见附录），从用例规约的自然语言描述中提取核心概念，并映射为领域对象。

识别与映射过程推导：

识别出的短语（	应用分类规则	建模决策
"线性规划模型"	[规则 1] 独立存在的实物/概念	识别为类 LPQModel
"包含" (Contains)	[规则 7] 表达包含/组成关系	识别为聚合关系（菱形）
"目标函数"、"约束"	[规则 2] 事物的组成部分	识别为类（作为模型的子部件）
"决策变量"	[规则 1] 被引用的独立概念	识别为类 DecisionVariable
"系数" (Coefficient)	[规则 3] 事物的特性/数值	识别为属性 (Attribute)

建模总结：

这一设计实现了**关注点分离**，将复杂的数学问题拆解为高内聚的独立对象（模型、函数、变量），避免了“上帝类”的出现，为后续分层架构奠定基础。



5.4 为类分配职责（GRASP 原则）

基于用例场景，运用 **GRASP 原则**对类进行行为建模。设计中采用**控制器模式**处理系统事件，并应用**信息专家与创建者模式**管理数据的生命周期。

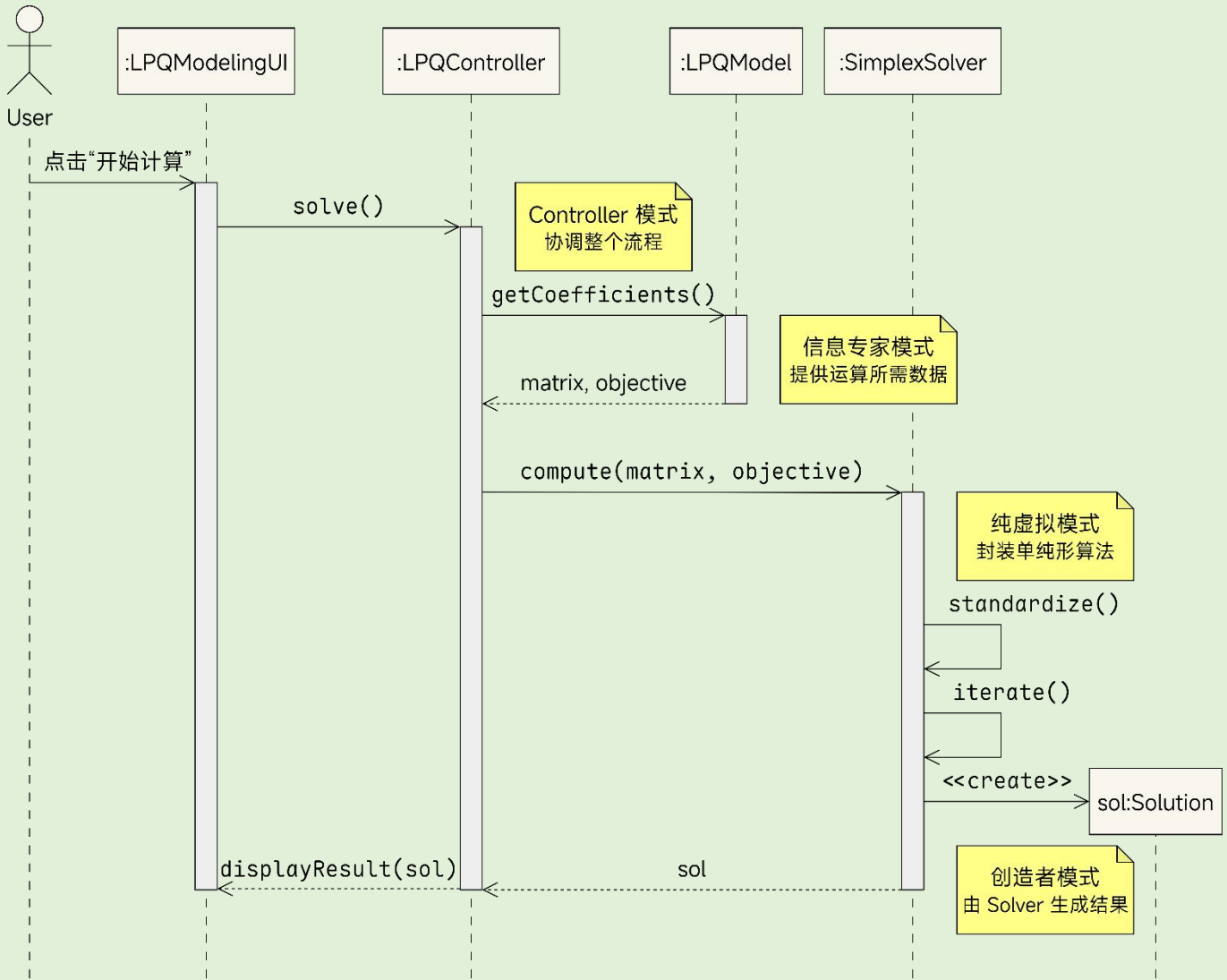
为避免实体类臃肿，特别引入**纯虚构模式（Pure Fabrication）**，将算法封装为独立的求解器对象。这一设计清晰定义了对对象协作，确保系统符合**高内聚、低耦合**的原则。

步骤	业务逻辑描述	应用 GRASP 模式	职责分配决策（类：方法）
1	接收用户点击“开始计算”的请求，协调后续流程	Controller (控制器)	LPQController:solve()
2	创建具体的变量对象 (Variable) 和约束对象 (Constraint)	Creator (创建者)	LPQModel:addVariable(),addConstraint() (因为模型包含变量)
3	获取目标函数的系数、约束条件的右端值	Information Expert (信息专家)	LPQModel:getCoefficients() (因为它拥有这些数据)
4	执行复杂的单纯形法运算 (Simplex Algorithm)	Pure Fabrication (纯虚构)	SimplexSolver:compute() (将算法独立，避免模型类臃肿)
5	封装最终的计算结果 (最优解、变量值)	Creator (创建者)	SimplexSolver:new Solution()

5.5 交互图设计（设计顺序图）

基于 GRASP 分配结果，从“黑盒”转向“白盒”视角，揭示对象间的消息传递机制。

- 1. UI → Controller: 捕获请求，委派处理。
- 2. 获取数据（信息专家）：Controller 向 Model 请求计算参数。
- 3. 执行算法（纯虚构）：Controller 调用独立求解器 SimplexSolver。
- 4. 封装结果（创建者）：求解器运算完成后，生成 Solution 对象返回。



“计划求解”详细设计顺序图

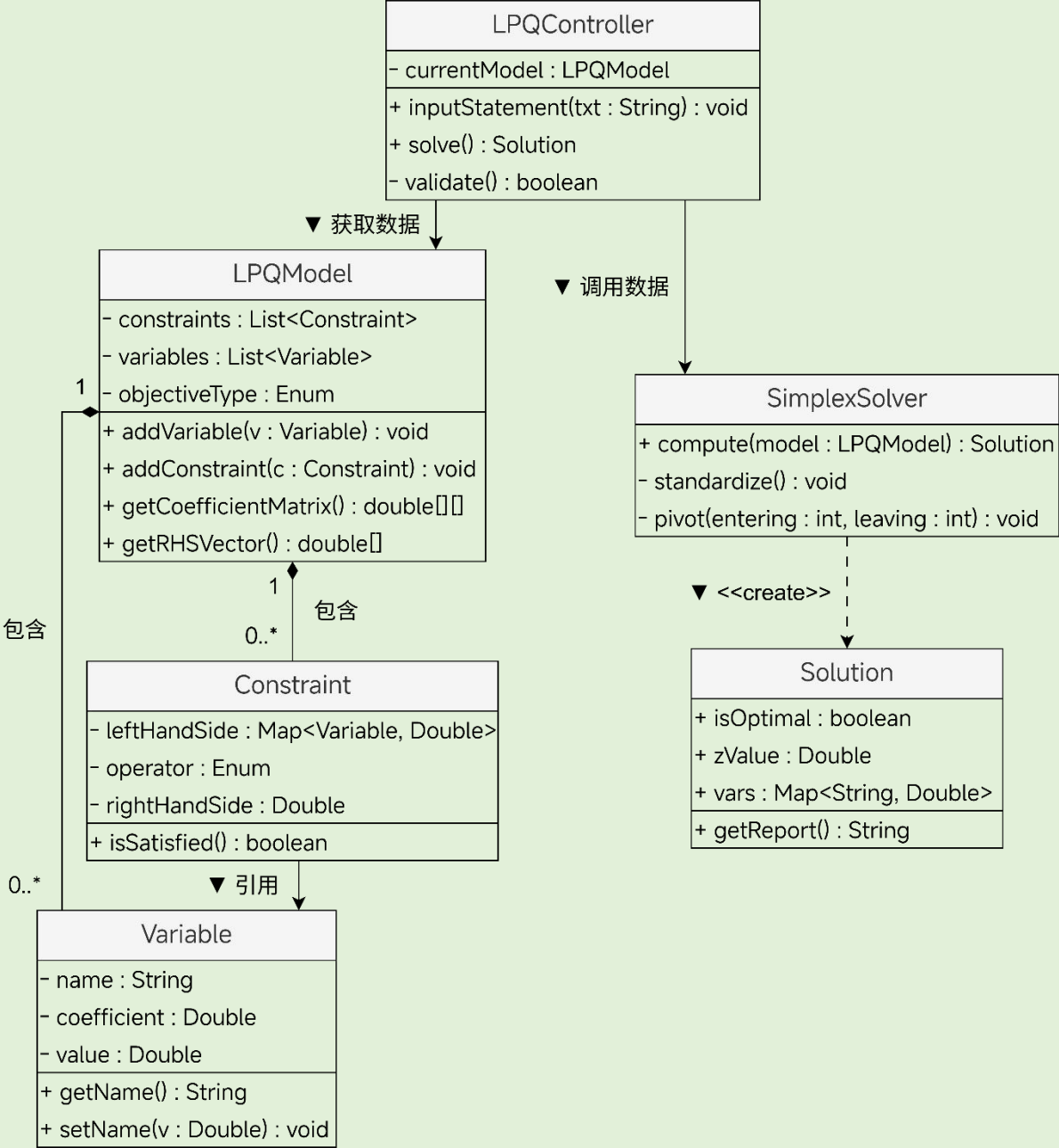
5.6 设计类图

基于交互图中的消息传递，将动态行为映射为静态的类方法（Methods），构建了最终的设计类图。

与之前的领域模型相比，设计类图增加了以下工程细节：

- 1. **方法定义**：将交互图中的 solve()、compute() 等消息落实为类的成员方法。
- 2. **类型规范**：明确了属性和参数的数据类型（如 double[][]）。
- 3. **导航性**：根据对象间的调用关系，确定了类之间的关联方向与可见性。

形成了一份可直接指导编码的详细蓝图，完成了从分析到设计的最终跨越。



LPQ 详细设计类图

5.7 算法详细设计

针对核心类 SimplexSolver 的 compute() 方法，完成了**算法级详细设计**。

完整描述了单纯形法的**三大核心步骤**：

1. **标准化**：引入松弛变量，构造初始单纯形表。
2. **最优性检验**：检查检验数是否全部非负。
3. **迭代优化**：执行基变换（选入基/出基变量）与高斯消元（Pivot）。

```
Algorithm SimplexSolver::compute(Model model)
// 数据预处理
(A, b, c) = convertToStandardForm(model) // 转为标准型
tableau = constructTableau(A, b, c) // 构造初始单纯形表

// 迭代主循环
while (true) do
    // 最优化检验
    if (all reduced_costs  $\geq$  0) then
        // 所有检验数非负，找到最优解
        return extractSolution(tableau)
    end if

    // 选择入基变量 (Bland 规则)
    enteringCol = findEnteringVariable()

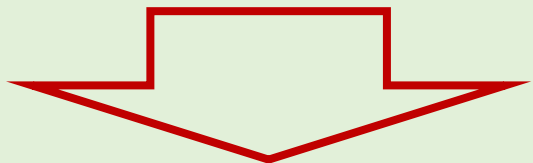
    // 无界解判定
    if (all cells in enteringCol  $\leq$  0) then
        throw UnboundException // 异常：问题无界
    end if

    // 选出基变量 (最小比值原则)
    leavingRow = findLeavingVariable(enteringCol)

    // 高斯消元 (旋转运算)
    pivot(leavingRow, enteringCol)

end while
End Algorithm
```

第 6 步：代码映射



6 从设计到实现

SimplexSolver
- tableau : double[][]
+ compute(model : LPQModel) : Solution
- standardize() : void
- pivot(row : int, col : int) : void

严格遵循软件工程的**正向工程**
(Forward Engineering) 原则，将设计模型转化为 C# 代码：

1. **结构映射**：UML 定义 → C# 类结构
2. **逻辑映射**：伪代码流程 → 控制流语句
3. **封装映射**：可见性 (-/+) → 修饰符 (private/public)

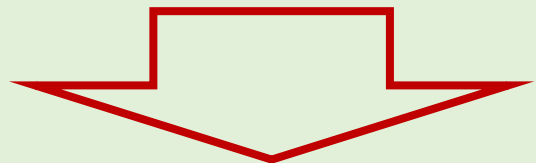
```

using System;

public class SimplexSolver
{
    private double[,] _tableau;
    public Solution Compute(LPQModel model)
    {
        // 标准化 (Standardize)
        Standardize(model);
        while (true)
        {
            // 最优性检验 (Optimality Test)
            if (CheckOptimality())
            {
                return new Solution(_tableau);
            }
            // 迭代优化 (Iteration)
            int enteringCol = FindEnteringVariable();
            int leavingRow = FindLeavingVariable(enteringCol);
            // 高斯消元
            Pivot(leavingRow, enteringCol);
        }
    }
    private void Pivot(int row, int col)
    {
        // C# 具体实现逻辑
    }
}

```

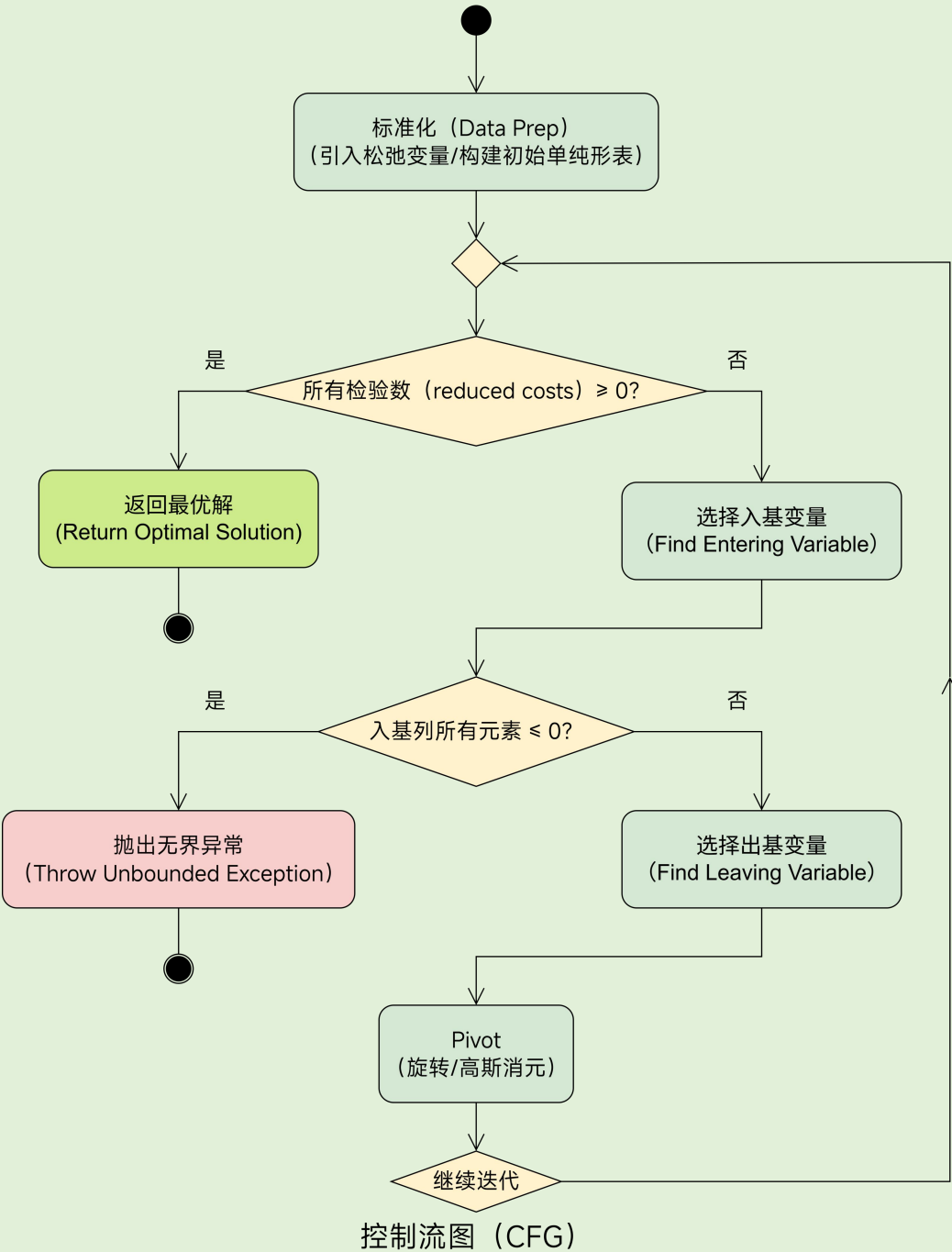
第 7 步：软件测试



7.1 单元测试（核心算法验证）

1. 白盒测试（White-box Testing）：基于详细设计的控制流图进行基础路径分析。
2. 等价类划分 & 边界值分析：针对判定节点（菱形框）设计测试数据。
 - 有效等价类：标准 LP 问题（TC-003）。
 - 无效/异常类：无界解（TC-002）。
 - 边界值：初始即最优（TC-001）、退化解（TC-004）。

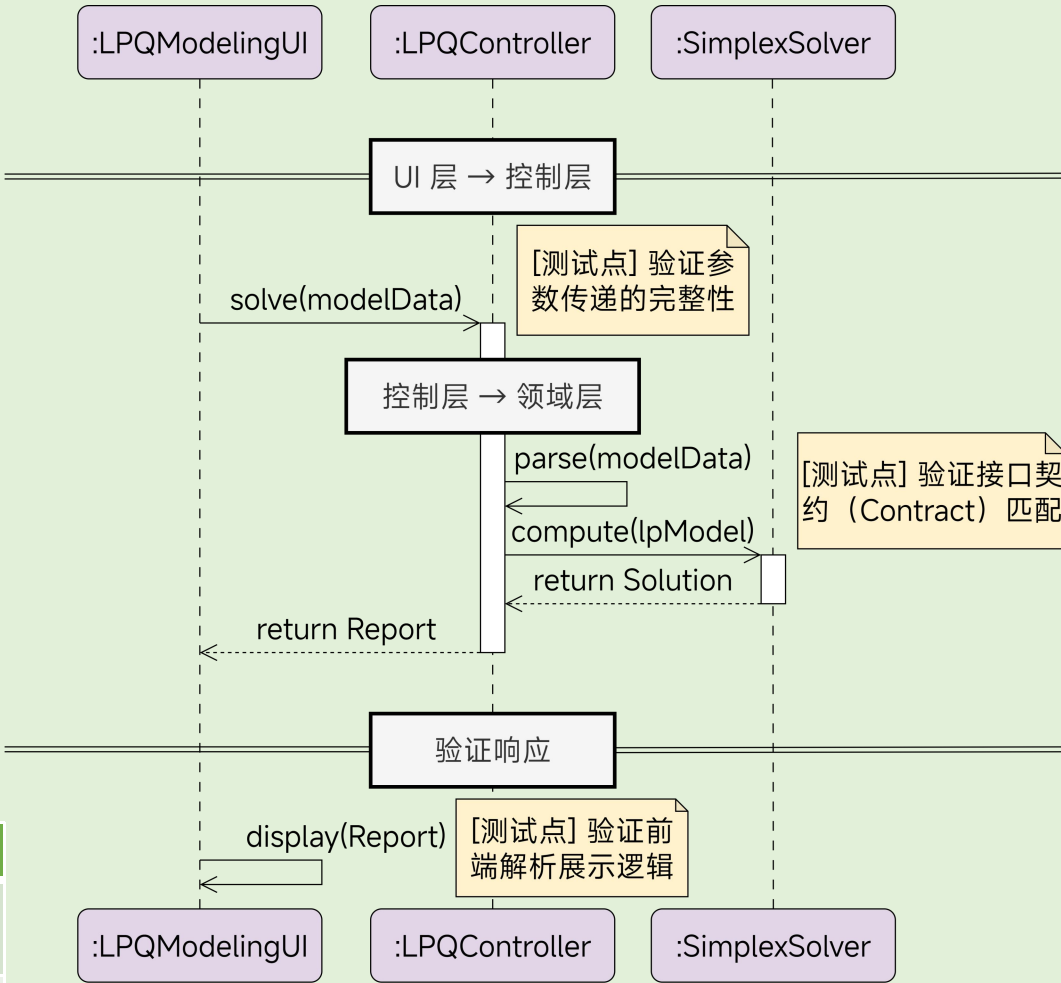
测试用例编号	覆盖路径	输入数据（LP 模型）	预期输出	实测结果
TC-001	路径 1 (直接最优)	$\max Z = -x_1 - x_2$ (所有系数为负，初始即最优)	Status: Optimal $Z = 0$	✓ Pass
TC-002	路径 2 (无界解)	$\max Z = x_1$ s. t. $x_1 \geq 5$ (无上界约束)	Status: Unbounded (或抛出异常)	✓ Pass
TC-003	路径 3 (正常迭代)	$\max Z = 2x_1 + x_2$ s. t. $x_1 + x_2 \leq 5$ (标注的课题例题)	Status: Optimal $Z = 10, x_1 = 5$	✓ Pass
TC-004	边界值 (退化/循环)	$\max Z = x_1$ s. t. $x_1 \leq 0, x_1 \geq 0$	Status: Optimal 正确结果	✓ Pass



7.2 集成测试（接口与交互验证）

- 1. 增量式集成（Incremental Integration）：采用“自顶向下”策略，先测试 UI 与 Controller 的交互，再测试 Controller 与 Solver 的连接。
- 2. 接口测试（Interface Testing）：重点验证层与层之间的方法调用与数据传递。
 - UI -> Controller：验证用户输入是否正确封装为 DTO 对象。
 - Controller -> Solver：验证业务对象（LPQModel）是否正确转化为算法矩阵。
- 3. 桩模块（Stub）：在测试 UI 层时，使用模拟的 Solver（Stub）返回固定数据，隔离算法干扰。

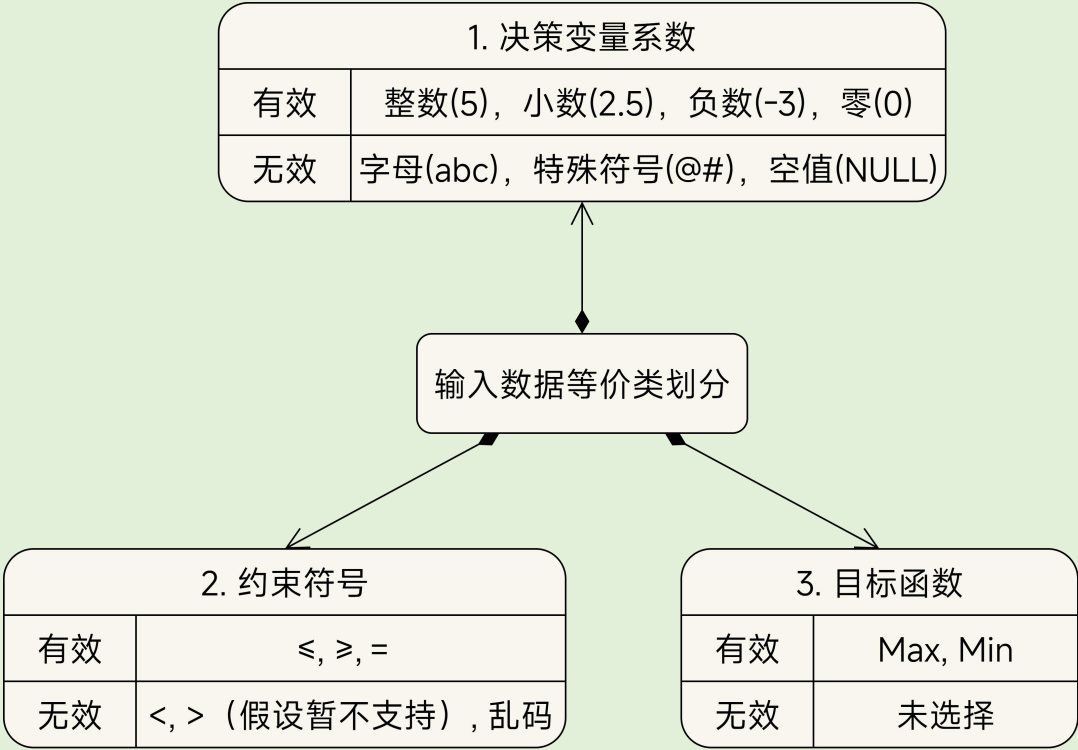
测试编号	集成路径	测试重点	桩模块（Stub）	结果
ITC-001	UI → Controller	验证输入数据包的封装与解析	Mock Solver（模拟算法返回）	✓ Pass
ITC-002	Controller → Solver	验证模型对象到矩阵的转换逻辑	无（真实调用）	✓ Pass
ITC-003	Client → Server	（分布式特有） 验证网络通信与序列化	无	✓ Pass



集成测试场景：全链路调用（ITC-001）

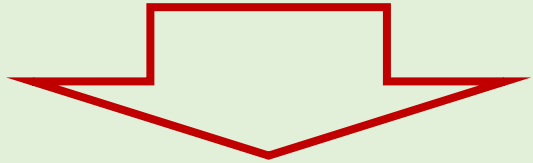
7.3 系统测试（基于等价类划分）

- 1. 黑盒测试（Black-box Testing）：不考虑内部代码，关注系统输入与输出的正确性。
- 2. 等价类划分（Equivalence Partitioning）：
 - 有效等价类：合法的数字输入（整数、小数、负数），验证业务计算正确性。
 - 无效等价类：非法的格式输入（字母、空值、特殊符号），验证系统的容错性与智能纠错机制（对应非功能需求）。
- 3. UI 交互测试：验证“建模向导”流程的连贯性。



测试编号	测试场景	输入数据	等价类类型	预期接货	实测结果
SYS-001	标准业务流	目标: Max 变量: $x_1 = 2.5, x_2 = -4$ 约束: ≤ 100	有效等价类 (小数 + 负数)	计算成功, 返回最优解 生成规划报告	✓ Pass
SYS-002	非法字符拦截	变量系数输入框填写: $2x + abc$	无效等价类 (非数字字符)	输入框变红 提示: “检测到非法字符”	✓ Pass
SYS-003	空值/未完成	定义了 3 个变量 但只填了 2 个系数就点计算	无效等价类 (数据缺失)	按钮置灰或弹窗提示 “请补全模型参数”	✓ Pass

总结



本次单纯形法计算工具的开发实践，对我而言不仅是一次代码编写的训练，更是一次完整的、标准化的软件工程全生命周期演练。从最初的软件发现、背景调研、开发策划，到中期的软件设计、编程实现，再到后期的系统测试与迭代优化，我亲历了软件从无到有的全过程，成功交付了一款满足高效性与安全性需求的分布式求解系统。

在技术实现上，我严格遵循了**“分而治之”**的工程思想。针对多用户并发需求，我采用了**分布式 C/S 架构**，并将系统划分为 UI 层、控制层、领域层与技术服务层，实现了清晰的关注点分离。在详细设计阶段，我深入运用 **GRASP 职责分配原则**，特别是通过引入**“纯虚构”模式**构建独立的求解器，成功避免了臃肿的“上帝类”出现，确保了系统的高内聚与低耦合。

最终，通过严格的**正向工程**实践，我确保了源码与设计模型的高度一致性。这一过程带给我最大的收获是工程思维的质变——我摒弃了以往“直接编码（Code-and-Fix）”的低效习惯，转而确立了**“设计先行（Design-First）”**的理念，深刻理解了建模不仅仅是绘图，更是控制软件复杂度、保证系统健壮性的核心手段。